УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА

Трг Доситеја Обрадовића 6, 21000 Нови Сад, Република Србија
Деканат: 021 6350-413; 021 450-810; Централа: 021 485 2000
Рачуноводство: 021 458-220; Студентска служба: 021 6350-763
Телефакс: 021 458-133; e-mail: ftndean@uns.ac.rs

ИНТЕГРИСАНИ СИСТЕМ МЕНАЏМЕНТА СЕРТИФИКОВАН ОД:

DOKUMENTACIJA ZA PREDMET:
Projektovanje elektronskih uređaja na sistemskom nivou

TEMA PROJEKTA:

ESL modelovanje i hardverska akceleracija algoritma senzorne fuzije IMU podataka (Madgwick IMU filter)

PROJEKAT IZRADILI:

Jelena Klisurić, EE62/2021
Luka Kojić, EE16/2022
Aleksandra Vukašinović, EE2/2022

MENTOR:

MSc Nebojša Pilipović

U Novom Sadu, 2026.

Sadržaj

1. Specifikacija **3**
1.1 Uvodni deo 3
1.2 Glavne funkcije 4
1.2.1 fast_inv_sqrt 4
1.2.2 AhrsHW_Sections5to8 4
1.3 Opis algoritma 5
2. Profajliranje **8**
3. Bitska analiza **10**
3.1 Tipovi fiksne tačke 10
3.2 Obrazloženje kritičnih izbora 12
3.2.1 quat_t - suženo sa 29 na 18 bita 12
3.2.2 dt_t - zadržano na 20 bita 12
3.2.3 dq_t - zadržano na 27 bita 13
3.2.4 fast_inv_sqrt - ostaje u pokretnom zarezu 13
3.3 Verifikacija 14
4. SW/HW particionisanje **16**
5. Virtuelna platforma **17**
5.1 Funkcionisanje virtuelne platforme 19
5.2 AHRS IP modul 20
6. Mešovita simulacija 23
6.1 Razlozi za odstupanje od klasičnog pristupa 23
6.2 Realizacija kosimulacije 23
6.3 Korišćeni fajlovi 25
6.4 Verifikaciona pokrivenost 26
7. Procena performansi **27**
7.1 Postupak HLS-a 28
Slika 9. Rezultati HLS-a u Vitisu 28
8. Literatura **33**

1. Specifikacija

1.1 Uvodni deo

IMU senzori danas imaju ključnu ulogu u uređajima kao što su dronovi, roboti, VR/AR sistemi i nosive tehnologije. Oni mere ubrzanje i ugaone brzine, ali zbog šuma i drifta ti podaci ne mogu direktno da se koriste za određivanje orijentacije. Zbog toga se primenjuju algoritmi senzorne fuzije, među kojima se Madgwick AHRS filter izdvojio kao jednostavan, brz i dovoljno precizan za embedded primenu.

Algoritam korišćen u ovom projektu zasnovan je na revidiranoj verziji opisanoj u poglavlju 7 doktorske disertacije Sebastijana Medžvika, dostupnoj kao otvoreni projekat Fusion (github.com/xioTechnologies/Fusion). Prema opisu repozitorijuma, algoritam funkcioniše kao komplementarni filtar koji kombinuje visoko-propusno filtrovana merenja žiroskopa sa nisko-propusno filtriranim merenjima akcelerometra, a izlaz je kvaternion orijentacije senzora u odnosu na Zemlju.

Softverska implementacija algoritma pri višim frekvencijama osvežavanja opterećuje procesor, što dovodi do većeg kašnjenja i povećane potrošnje energije. Rešenje je prebacivanje najzahtevnijih operacija - normalizacije vektora, inverznog kvadratnog korena i kvaternion aritmetike - na namenski hardver. Implementacija je organizovana kao podela na softverski deo (main.cpp, odeljci 1-4 i 9-11) i hardverski deo (ahrs_hw.cpp, odeljci 5–8), predviđen za realizaciju na FPGA platformi Zybo Z7-10. Obrada se vrši uzorak po uzorak pri učestanosti od 100 Hz (SAMPLE_RATE = 100).

U industriji postoji izražen interes za ovakvim rešenjima - od proizvođača dronova koji zahtevaju brže reakcije upravljanja, do VR/AR kompanija i robotike koja zahteva stabilne orijentacione informacije u realnom vremenu. Konačni cilj je pouzdan hardverski modul koji ubrzava Madgwick algoritam i smanjuje opterećenje procesora.

1.2 Glavne funkcije

1.2.1 fast_inv_sqrt

Funkcija za brzo izračunavanje inverznog kvadratnog korena, implementirana u ahrs_hw.cpp. Ovo je najkritičnija aritmetička operacija u algoritmu - poziva se do tri puta po uzorku, a Fusion repozitorijum eksplicitno navodi primenu Pizerovog algoritma za normalizaciju vektora i kvaterniona. Implementacija koristi bit-manipulaciju nad IEEE 754 reprezentacijom: magična konstanta 0x5F1F1412 daje početnu aproksimaciju, a zatim se primenjuje jedna iteracija Njutn-Rafsonove metode:

```
u.i = 0x5F1F1412 - (u.i >> 1);
return u.f * (1.69000231f - 0.714158168f * x * u.f * u.f);
```

U odeljku 5 poziva se uvek jednom za normalizaciju akcelerometra (INV_SQRT #1), i uslovno drugi put za normalizaciju vektorskog produkta kada je skalarni proizvod negativan (INV_SQRT #2). Iste konstante koriste se i u softverskom odeljku 9 za normalizaciju izlaznog kvaterniona (main.cpp, FusionAhrsUpdate).

1.2.2 AhrsHW_Sections5to8

Funkcija koja čini hardverski akcelerator, implementirana u ahrs_hw.cpp. Pokriva odeljke 5–8: povratnu spregu akcelerometra sa rejection logikom, konverziju žiroskopa, primenu povratne sprege i integraciju kvaterniona. Prima AHRS state, merenja žiroskopa (°/s) i akcelerometra (g), vektor pola gravitacije (halfGravityX/Y/Z) iz softverskog odeljka 4, i vremenski korak deltaTime. Vraća nenormalizovani kvaternion po izlaznim pokazivačima (newQw/newQx/newQy/newQz).

Ključna novina u ovoj verziji je rejection logika u odeljku 5: nakon izračunavanja vektora povratne sprege, proverava se kvadrat njegove norme (feedbackNormSq). Ako je sistem u inicijalizaciji ili je feedbackNormSq manji od praga accelerationRejection, akcelerometar se prihvata (trigger −= 9); inače trigger += 1. Trigger se steže na [0, recoveryTriggerPeriod], a vektor halfAccelFeedback primenjuje se samo ako accelerometerIgnored == false.

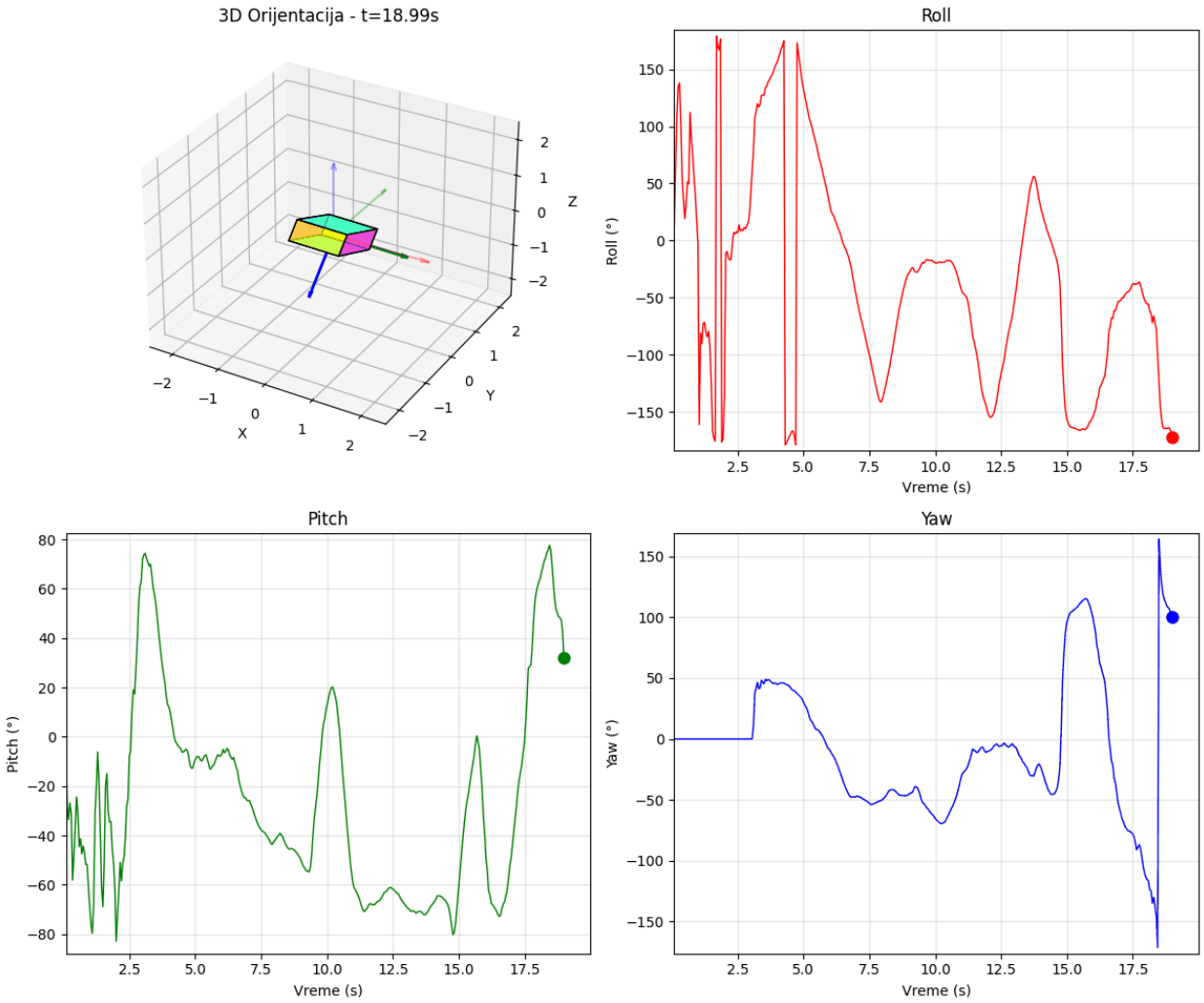
1.3 Opis algoritma

Program za estimaciju orijentacije IMU senzora sastoji se iz sledećih celina.

Softverski deo napisan je u programskom jeziku *C++*, i obuhvata kalibraciju ulaza, korekciju pomaka žiroskopa i odeljke 1–4 te 9–11 algoritma (main.cpp). Ulazni podaci mogu biti CSV ili binarni fajl - format se detektuje automatski po ekstenziji (.csv ili .bin).

Hardverski deo, modelovan funkcijom *AhrsHW_Sections5to8* (ahrs_hw.cpp), implementira odeljke 5–8 i predviđen je za realizaciju u VHDL-u na FPGA platformi Zybo Z7-10.

Skripta za vizualizaciju napisana je u *Python* programskom jeziku (visualize_motion.py). Ovaj deo koda nije uključen u merenje performansi.



Slika 1. Vizualizacija napisana u Python programskom jeziku

Sam algoritam obrađuje uzorke jedan po jedan i sastoji se iz sledećih segmenata:

1. **Kalibracija senzora** - funkcija *FusionCalibrationInertial* primenjuje model result = misalignment * (uncalibrated − offset) * sensitivity odvojeno za žiroskop i akcelerometar. U referentnoj implementaciji parametri su neutralni (identity matrica, jedinična osetljivost, nulti pomak), što obezbeđuje infrastrukturu bez numeričkog efekta na rezultat (main.cpp, run_pipeline).
2. **Korekcija pomaka žiroskopa** - funkcija *FusionOffsetUpdate* akumulira estimat pomaka tokom mirnih perioda (sve ose < ±3°/s tokom 500 uzoraka) sa koeficijentom 0.01f i oduzima ga od svakog merenja (main.cpp, run_pipeline).
3. **Odeljak 1** - Čuvanje akcelerometra: vrednosti ax/ay/az upisuju se u ahrs->accelerometer (main.cpp, FusionAhrsUpdate).
4. **Odeljak 2** - Provera opsega žiroskopa: ako bilo koja osa premaši 2000°/s, vrši se puni restart internog stanja - sačuva se kvaternion, a resetuju se rampedGain = 10.0f, initialising = true, halfAccelerometerFeedback = {0,0,0}, accelerometerIgnored = false i accelerationRecoveryTrigger = 0 (main.cpp, FusionAhrsUpdate).
5. **Odeljak 3** - Postepeno smanjenje pojačanja: rampedGain se smanjuje za rampedGainStep * deltaTime tokom inicijalizacije, gde je rampedGainStep = (10.0 − 0.5) / 3.0. Kada padne ispod gain = 0.5f, initialising se postavlja na false (main.cpp, FusionAhrsUpdate).
6. **Odeljak 4** - Vektor pola gravitacije iz tekućeg kvaterniona {qw, qx, qy, qz}:
   halfGravityX = qx*qz − qw*qy
   halfGravityY = qy*qz + qw*qx
   halfGravityZ = qw*qw − 0.5 + qz*qz
   Prosleđuje se HW akceleratoru kao referenca (main.cpp, FusionAhrsUpdate).
7. **Odeljak 5 (HW)** - Povratna sprega akcelerometra sa rejection logikom: normalizacija akcelerometra (INV_SQRT #1), vektorski produkt sa halfGravity, uslovna normalizacija produkta ako je dotProduct < 0 (INV_SQRT #2), i rejection logika opisana u sekciji 1.2.2 (ahrs_hw.cpp, AhrsHW_Sections5to8).
8. **Odeljak 6 (HW)** - Konverzija žiroskopa: merenja u °/s množe se konstantom degToRadHalf = π / 360.0f (konverzija u rad/s i množenje sa 0.5 za kvaternionsku matematiku) (ahrs_hw.cpp, AhrsHW_Sections5to8).
9. **Odeljak 7 (HW)** - Primena povratne sprege:
   adjHalfGyro = halfGyro + halfAccelFeedback * rampedGain(ahrs_hw.cpp, AhrsHW_Sections5to8).

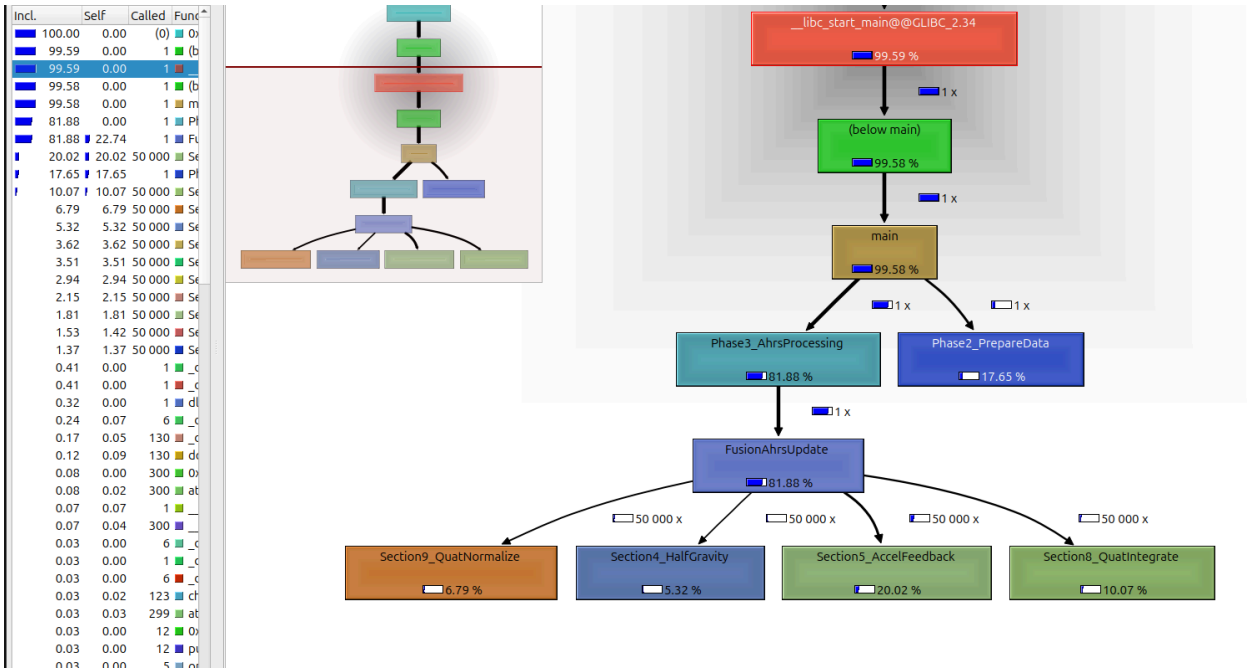
10. **Odeljak 8 (HW)** - Integracija kvaterniona: izvod kvaterniona i Ojlerova integracija q_new = q + dq * deltaTime, gde su komponente izvoda:
    dqw = −qx*adjX − qy*adjY − qz*adjZ
    dqx = qw*adjX + qy*adjZ − qz*adjY
    dqy = qw*adjY − qx*adjZ + qz*adjX
    dqz = qw*adjZ + qx*adjY − qy*adjX
    Izlaz je nenormalizovani kvaternion (ahrs_hw.cpp, AhrsHW_Sections5to8).
11. **Odeljak 9 (SW)** - Normalizacija kvaterniona: primenom fast_inv_sqrt (konstanta 0x5F1F1412, jedan Newton-Raphson korak) rezultat se normalizuje i upisuje u ahrs->quaternion (main.cpp, FusionAhrsUpdate).
12. **Odeljak 10 (SW)** - Nulovanje azimuta, aktivno samo tokom inicijalizacije: iz tekućeg kvaterniona izračunava se yaw, formira rotacioni kvaternion oko ose Z
    (cosHY = cos(yaw/2), sinHY = sin(yaw/2)) i množi sa tekućim kvaternionom (main.cpp, FusionAhrsUpdate).
13. **Odeljak 11 (SW)** - Upis izlaza: konvertovani kvaternion se upisuje u izlazni CSV fajl, odnosno u DDR memoriju u kontekstu virtuelne platforme (main.cpp, run_pipeline).

Po završetku svakog poziva funkcije *FusionAhrsUpdate*, normalizovani kvaternion se nalazi u ahrs->quaternion i odmah se konvertuje u Ojlerove uglove funkcijom *FusionEulerFrom* i upisuje u izlazni CSV fajl (main.cpp, run_pipeline).

2. Profajliranje

U cilju identifikacije računski najzahtevnijih delova algoritma, izvršeno je profajliranje referentne C implementacije Madgwick AHRS filtera. Tokom analize obrađeno je 50 000 uzoraka senzorskih podataka. Profilisanje je sprovedeno korišćenjem alata KCachegrind.

Na Slici 1 prikazan je pregled izvršavanja algoritma po fazama. Može se uočiti da se najveći deo vremena provodi u funkciji *FusionAhrsUpdate*, koja zauzima **81.88%** ukupnog vremena u okviru faze *Phase3_AhrsProcessing*. Faza *Phase2_PrepareData* učestvuje sa 17.65% i obuhvata pripremu ulaznih podataka.



Slika 2. Izvrsavanje algoritma po fazama

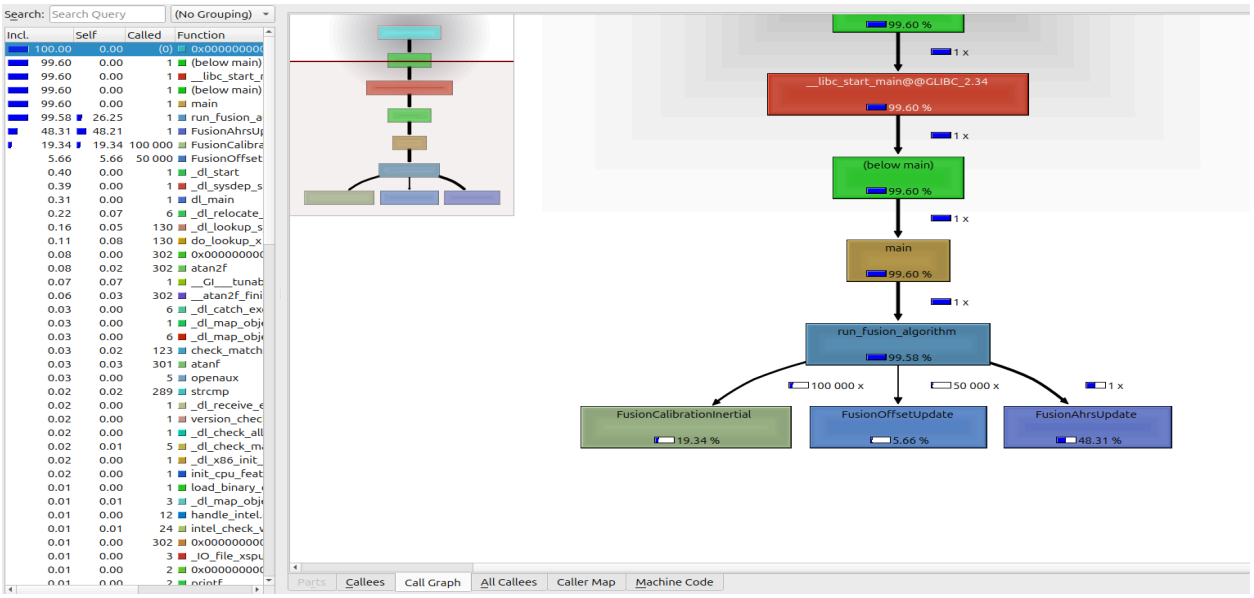
U tabeli na Slici 3 je prikazana detaljna raspodela:

99.58	0.00	1	main	ahrs_callgrind: ahrs_callgrind_profile.c
81.88	0.00	1	Phase3_AhrsProcessing	ahrs_callgrind: ahrs_callgrind_profile.c
81.88	22.74	1	FusionAhrsUpdate	ahrs_callgrind: ahrs_callgrind_profile.c
20.02	20.02	50 000	Section5_AccelFeedback	ahrs_callgrind: ahrs_callgrind_profile.c
17.65	17.65	1	Phase2_PrepareData	ahrs_callgrind: ahrs_callgrind_profile.c
10.07	10.07	50 000	Section8_QuatIntegrate	ahrs_callgrind: ahrs_callgrind_profile.c
6.79	6.79	50 000	Section9_QuatNormalize	ahrs_callgrind: ahrs_callgrind_profile.c
5.32	5.32	50 000	Section4_HalfGravity	ahrs_callgrind: ahrs_callgrind_profile.c
3.62	3.62	50 000	Section2_GyroRangeCh...	ahrs_callgrind: ahrs_callgrind_profile.c
3.51	3.51	50 000	Section7_ApplyFeedback	ahrs_callgrind: ahrs_callgrind_profile.c
2.94	2.94	50 000	Section6_GyroConvert	ahrs_callgrind: ahrs_callgrind_profile.c
2.15	2.15	50 000	Section1_StoreAccel	ahrs_callgrind: ahrs_callgrind_profile.c
1.81	1.81	50 000	Section11_Output	ahrs_callgrind: ahrs_callgrind_profile.c
1.53	1.42	50 000	Section10_ZeroHeading	ahrs_callgrind: ahrs_callgrind_profile.c
1.37	1.37	50 000	Section3_RampGain	ahrs_callgrind: ahrs_callgrind_profile.c

Slika 3. Rezultati profajliranja - raspodela vremena po funkcijama

Detaljniji prikaz (Slika 4) pokazuje raspodelu vremena unutar funkcije *FusionAhrsUpdate*.
Sve ključne sekcije izvršavaju se 50 000 puta, a najznačajnije među njima su:

- **Section5_AccelFeedback — 20.02%**
- **Section8_QuatIntegrate — 10.07%**
- **Section9_QuatNormalize — 6.79%**



Slika 4. Raspodela vremena unutar FusionAhrsUpdate

Dodatni prikaz (Slika 4) na nivou funkcije **run_fusion_algorithm** potvrđuje da *FusionAhrsUpdate* predstavlja dominantan deo izvršavanja (48.31%), dok funkcije FusionCalibrationInertial (19.34%) i FusionOffsetUpdate (5.66%) imaju manji značaj i nisu deo kritične putanje u stabilnom radu sistema.

Na osnovu ovih rezultata, sekcije 5–8 su identifikovane kao kandidati za hardversku akceleraciju, jer zajedno čine dominantni deo računskog opterećenja, izvršavaju se 50 000 puta i ne sadrže zavisnosti koje bi sprečile njihovu implementaciju u hardveru.

3. Bitska analiza

Promenljive u C++ implementaciji algoritma (sekcije 5–8) su realni brojevi u pokretnom zarezu. Za hardversku realizaciju na FPGA platformi neophodno ih je predstaviti tipovima sa fiksnom tačkom koji pokrivaju stvarni opseg vrednosti uz prihvatljiv gubitak tačnosti.

Tipovi su određeni iterativno: pokretanjem algoritma za različite kombinacije broja celobrojnih i razlomljenih bita, pri čemu su praćeni i međurezultati, a ne samo krajnja orijentacija. Sve promenljive koriste zaokruživanje SC_RND i saturaciju SC_SAT. Definicije tipova nalaze se u fajlu analysis/spec/src/ahrs_hw.cpp.

3.1 Tipovi fiksne tačke

Za svaku promenljivu algoritma određeni su opseg vrednosti, ukupan broj bita, broj bita za celobrojni deo i broj bita za razlomljeni deo.

Promenljiva	Tip SystemC	Opseg	Ukupno bita	Cel. bita	Razl. bita	Napomena
`quat_t`	`sc_fixed`	[-2, 2)	18	2	16	suženo: 29 → 18
`dot_t`	`sc_fixed`	[-1, 1)	26	1	25	
`norm_t`	`sc_fixed`	[-2, 2)	26	2	24	
`accel_t`	`sc_fixed`	[-4, 4)	20	3	17	
`hg_t`	`sc_fixed`	[-2, 2)	26	2	24	
`magsq_t`	`sc_ufixed`	[0, 4)	22	2	20	
`invmag_t`	`sc_ufixed`	[0, 64)	24	6	18	
`halfgyro_t`	`sc_fixed`	[-2, 2)	22	2	20	
`gain_t`	`sc_ufixed`	[0, 16)	20	4	16	
`adjhg_t`	`sc_fixed`	[-16, 16)	26	5	21	
`dq_t`	`sc_fixed`	[-16, 16)	27	5	22	zadržano: 27 bita
`dt_t`	`sc_ufixed`	[0, 1)	20	0	20	zadržano: 20 bita

Tabela 1. Ukupan broj bita, broj bita za celobrojni deo i broj bita za razlomljeni deo

```
// Quaternion [-2, 2) → SMANJENO: 29→18 bita, 16 frac
// quat(18) × adjhg(26) = 44-bit ≤ 45 → staje u 1 DSP48E1 ✓
typedef sc_fixed<18, 2, SC_RND, SC_SAT>  quat_t;

// Dot product [-1, 1) → 25 frac (26-bit, nepromenjen)
typedef sc_fixed<26, 1, SC_RND, SC_SAT>  dot_t;

// Normalizovani vektori [-2, 2) → 24 frac (26-bit, nepromenjen)
typedef sc_fixed<26, 2, SC_RND, SC_SAT>  norm_t;

// Akcelerometar [-4, 4) → 17 frac (20-bit, nepromenjen)
typedef sc_fixed<20, 3, SC_RND, SC_SAT>  accel_t;

// HalfGravity [-2, 2) → 24 frac (26-bit, nepromenjen)
typedef sc_fixed<26, 2, SC_RND, SC_SAT>  hg_t;

// Magnitude squared [0, 4) → 20 frac (22-bit, nepromenjen)
typedef sc_ufixed<22, 2, SC_RND, SC_SAT> magsq_t;

// Inverse magnitude [0, 64) → 18 frac (24-bit, nepromenjen)
typedef sc_ufixed<24, 6, SC_RND, SC_SAT> invmag_t;

// HalfGyro [-2, 2) → 20 frac (22-bit, nepromenjen)
typedef sc_fixed<22, 2, SC_RND, SC_SAT>  halfgyro_t;

// Gain [0, 16) → 16 frac (20-bit, nepromenjen)
typedef sc_ufixed<20, 4, SC_RND, SC_SAT> gain_t;

// AdjHalfGyro [-16, 16) → 21 frac (26-bit, nepromenjen)
typedef sc_fixed<26, 5, SC_RND, SC_SAT>  adjhg_t;

// dq [-16, 16) → 22 frac (27-bit, nepromenjen)
// dq(27) × dt(20) = 47-bit → 2-DSP kaskada (prihvatljivo)
typedef sc_fixed<27, 5, SC_RND, SC_SAT>  dq_t;

// DeltaTime [0, 1) → 20 frac (20-bit, OSTAJE jer 18-bit lomi preciznost za dt=0.01)
typedef sc_ufixed<20, 0, SC_RND, SC_SAT> dt_t;
```

Slika 5. Definicije tipova u kodu (analysis/spec/src/ahrs_hw.cpp)

3.2 Obrazloženje kritičnih izbora

3.2.1 quat_t - suženo sa 29 na 18 bita.

Ovo je najvažnija promena u konfiguraciji tipova. Razlog je ograničenje DSP bloka na ciljnom FPGA uređaju xc7z010 (Zybo Z7-10): blok podržava maksimalno množenje **18 × 27 = 45 bita.** U sekciji 8 nalaze se 12 množenja oblika quat × adjhg, što se direktno vidi u kodu:

```
const dq_t dqw = -qx * adjHalfGyroX - qy * adjHalfGyroY - qz *
adjHalfGyroZ;
const dq_t dqx =  qw * adjHalfGyroX + qy * adjHalfGyroZ - qz *
adjHalfGyroY;
const dq_t dqy =  qw * adjHalfGyroY - qx * adjHalfGyroZ + qz *
adjHalfGyroX;
const dq_t dqz =  qw * adjHalfGyroZ + qx * adjHalfGyroY - qy *
adjHalfGyroX;
```

Sa originalnih 29 bita za quat_t, svaki od tih proizvoda iznosi 29 + 26 = 55 bita, što premašuje kapacitet jednog DSP48E1 bloka i zahteva kaskadu od dva bloka po množenju. Sužavanjem quat_t na 18 bita, svaki proizvod postaje 18 + 26 = 44 bita, što staje u jedan DSP48E1 blok. To se eksplicitno navodi u zaglavlju koda:

```
// quat_t : 29→18 bita  →  quat(18) × adjhg(26) = 44-bit ≤ 45 → 1 DSP
```

Pošto u sekciji 8 postoje 12 takvih množenja, suženje quat_t eliminuje potrebu za 12 dodatnih DSP48E1 blokova u toj sekciji.

3.2.2 dt_t - zadržano na 20 bita.

Testiran je pokušaj suženja na 18 bita. Za vrednost deltaTime = 0.01 s (frekvencija uzorkovanja 100 Hz, SAMPLE_RATE = 100 u main.cpp), kvantizaciona greška pri 18-bitnoj reprezentaciji iznosi -1.68×10^{-5}. Tokom faze inicijalizacije, rampedGain se smanjuje od početnih 10.0 do nominalne vrednosti 0.5, korak po korak:

```
ahrs->rampedGain     = 10.0f;
ahrs->rampedGainStep = (10.0f - 0.5f) / 3.0f;   // 3.0f = period
inicijalizacije u sekundama
ahrs->rampedGain -= ahrs->rampedGainStep * deltaTime;  // pri svakom
uzorku
```

Inicijalizacija traje 3 sekunde, što pri 100 Hz odgovara 300 uzoraka. Greška od -1.68×10^{-5} se tokom inicijalizacije množi pojačanjem rampedGain ≈ 10 i propagira kroz kvaternionsku integraciju (Section8_QuatIntegrate) i naknadnu normalizaciju. Empirijski merena akumulirana greška Ojlerovih uglova na test skupu od 499 uzoraka iznosi ~30°, što je potpuno neprihvatljivo. Tip je stoga zadržan na 20 bita. Ovo je eksplicitno objašnjeno u zaglavlju koda:

```
// dt_t : OSTAJE 20-bit!  (18-bit daje kvantizacionu grešku dt=0.01 od
-1.68e-5,
//                         koja se množi gain≈10 × 499 uzoraka →
akumulira ~30° !)
```

3.2.3 dq_t - zadržano na 27 bita.

Testovane su kombinacije sužavanja dq_t na 22 ili 18 bita zajedno sa odgovarajućim vrednostima dt_t. U svim slučajevima greška na dugom skupu podataka prelazila je 30°. Množenje dq(27) × dt(20) = 47 bita premašuje kapacitet jednog DSP48E1 bloka i zahteva 2-DSP kaskadu, ali je to jedino rešenje koje daje prihvatljivu tačnost. Ovo je navedeno u zaglavlju koda:

```
// dq_t : OSTAJE 27-bit  →  dq(27) × dt(20) = 47-bit → 2-DSP kaskada
(prihvatljivo)
```

3.2.4 fast_inv_sqrt - ostaje u pokretnom zarezu.

Pozivi funkcije fast_inv_sqrt u sekciji 5 vrše se eksplicitnim kastovanjem fiksno-tačkastih promenljivih u float, a rezultat se dodeljuje tipu invmag_t:

```
const invmag_t accelInvMag = fast_inv_sqrt((float)accelMagSquared);
const invmag_t crossInvMag = fast_inv_sqrt((float)crossMagSquared);
```

Na isti način, rejection logika u sekciji 5 računa feedbackNormSq nad float vrednostima dobijenim kastovanjem iz fiksno-tačkastih promenljivih, jer se poređenje vrši sa ahrs->accelerationRejection koji je float tip iz AHRS state strukture.

3.3 Verifikacija

Implementacija sa fiksnom tačkom (analysis/spec/src/ahrs_hw.cpp) upoređena je sa float referentnom implementacijom (cpp_spec/spec/src/ahrs_hw.cpp) na dva skupa podataka IMU senzora.

Metrika	Vrednost
Kvaternion greška – maksimalna	0.30°
Kvaternion greška – srednja	0.12°
Kvaternion greška – 95. percentil	0.24°
Roll greška – maksimalna	0.30°
Pitch greška – maksimalna	0.10°
Yaw greška – maksimalna	0.24°
Uzoraka sa greškom < 0.5°	499 (100 %)

Tabela 2. Kratki skup: 499 uzoraka (≈ 5 sekundi pri 100 Hz)

Float referenca: Roll = 129.958°, Pitch = 9.287°, Yaw = 25.567°
Fixed-point: Roll = 129.987°, Pitch = 9.198°, Yaw = 25.353°

Metrika	Vrednost
Kvaternion greška – maksimalna	2.12°
Kvaternion greška – srednja	0.37°
Kvaternion greška – 95. percentil	1.91°
Kvaternion greška – 99. percentil	2.08°

Tabela 3. Dugi skup: 1986 uzoraka (≈ 20 sekundi pri 100 Hz)

Float referenca: Roll = −132.663°, Pitch = −28.216°, Yaw = 53.252°
Fixed-point: Roll = −133.468°, Pitch = −27.150°, Yaw = 54.718°

Opseg greške	Broj uzoraka	Procenat
< 0.5°	1745	87.9 %
0.5° – 1.0°	2	0.1 %
1.0° – 2.0°	179	9.0 %
2.0° – 5.0°	60	3.0 %
> 5.0°	0	0.0 %

Tabela 4. Raspodela kvaternion greške — dugi skup (1986 uzoraka)

Napomena o grešci Euler uglova na dugom skupu:

Na dugom skupu uočena je maksimalna greška Euler uglova od ~9° (roll i yaw). Ova greška je artefakt konverzije iz kvaterniona u Euler uglove i javlja se samo u uzorcima gde je pitch blizu 90° (uzorci 1820–1832, pitch ≈ 78°) - blizina gimbal-lock singulariteta. U toj zoni mali pomak kvaterniona matematički odgovara velikom pomaku roll/yaw uglova. Stvarna kvaternion greška u tim uzorcima iznosi 0.0°. Kvaternioni su identični u okviru preciznosti float reprezentacije u CSV izlaznom fajlu.

Odabrana konfiguracija tipova fiksne tačke zadovoljava oba ključna zahteva. Sa stanovišta tačnosti, maksimalna kvaternion greška iznosi 2.12° na skupu od 1986 uzoraka i 0.30° na skupu od 499 uzoraka, što je prihvatljivo za embedded IMU primene. Sa stanovišta DSP efikasnosti, sužavanje quat_t sa 29 na 18 bita smanjuje svako od 12 množenja quat × adjhg u sekciji 8 sa 55 na 44 bita, čime svako od tih množenja staje u jedan DSP48E1 blok umesto u kaskadu od dva.

4. SW/HW particionisanje

Na osnovu rezultata profajliranja u poglavlju 2, kandidati za hardversku akceleraciju identifikovani su kao sekcije sa najvećim udelom u izvršnom vremenu unutar funkcije FusionAhrsUpdate:

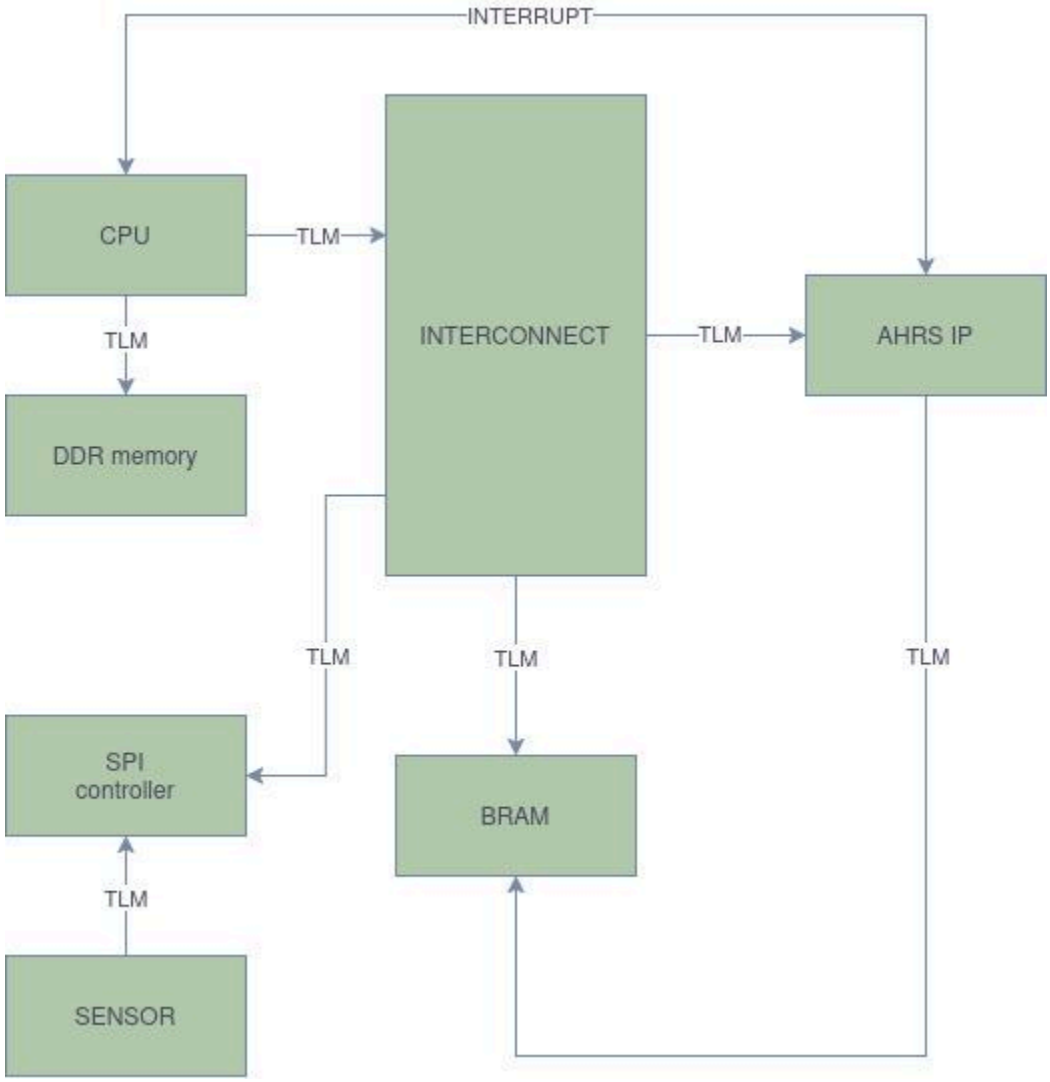
Section5_AccelFeedback (20.02%) — dominira vremenskim opterećenjem zbog dve invocacije fast_inv_sqrt funkcije i kvadratnih korena nad floating-point vrednostima.

Section8_QuatIntegrate (10.07%) — 12 multiplikacija oblika quat × adjhg i 4 multiplikacije sa deltaTime, idealna za pipeline implementaciju u DSP48E1 blokovima.

Section9_QuatNormalize (6.79%) — treća najteža sekcija po vremenu, ali zadržana u softveru iz razloga objašnjenog u sekciji 3.2.4: fast_inv_sqrt ostaje u pokretnom zarezu jer su magična konstanta 0x5F1F1412 i Njutn-Rafsonova iteracija optimizovane upravo za IEEE 754 reprezentaciju.

Sekcije 5–8 (zajedno ~36% vremena unutar FusionAhrsUpdate) particionisane su u hardver kao jedan blok jer dele zajedničke ulaze i međurezultate koji se direktno propagiraju kroz pipeline. Sekcije 1–4 i 10–11 ostaju u softveru jer obuhvataju kontrolnu logiku (rejection logika, ramping pojačanja, nulovanje azimuta) koja se ne ponavlja deterministički i ne bi profitirala od fiksne pipeline strukture.

5. Virtuelna platforma



Slika 6. Izgled virtuelne platforme (blok dijagram)

Na Slici 6 prikazana je blok šema virtuelne platforme. Virtuelna platforma je softverski model sistema koji služi kao zamena hardverskom prototipu u ranim fazama razvoja. Modeluje hardversko/softversko particionisanje Madgwick AHRS algoritma na Zybo Z7-10 razvojnoj ploči: softver (ARM Cortex-A9) izvršava sekcije 1–4 i 9–11 algoritma, dok hardverski akcelerator (AHRS IP) preuzima sekcije 5–8 u fixed-point aritmetici. Sistem radi po principu obrade uzorak po uzorak (sample-by-sample). Svaki IMU uzorak se samostalno obrađuje kroz ceo lanac pre nego što stigne sledeći. Ulaz u sistem su sirovi podaci IMU senzora (ugaone brzine i ubrzanja), a izlaz su normalizovani kvaternioni orijentacije upisani u DDR memoriju.

Virtuelna platforma se sastoji od sledećih modula:

- **SENSOR** - predstavlja IMU senzor koji kontinualno generiše sirove podatke (ugaone brzine i ubrzanja). Modelovan je kao izvor stimulus podataka koji pasivno čeka TLM read zahteve od SPI kontrolera - ne inicira ništa sam.
- **SPI kontroler** - čita podatke sa senzora i čuva ih u internom registru. Ponaša se kao target (slave) na magistrali sa sopstvenim adresnim opsegom (0x0300–0x031F). CPU ga adresira kroz Interconnect kao što bi čitao bilo koji AXI periferal. Nakon svakog preuzetog uzorka SPI notifikuje CPU putem globalnog događaja sensor_ready.
- **CPU** - jedini master (initiator) u sistemu - jedini koji inicira TLM transakcije prema Interconnect-u. Modeluje ARM Cortex-A9 koji živi u PS delu Zybo Z7-10. Za svaki pristigli uzorak čita SPI registre, priprema ulazne podatke za AHRS IP (SW sekcije 1–4), upisuje ih u BRAM, startuje AHRS IP, čeka interrupt, i na kraju normalizuje rezultate (SW sekcije 9–11) i upisuje ih u DDR.
- **DDR memorija** - privatna memorija CPU-a za čuvanje finalnih normalizovanih kvaterniona nakon obrade. Direktno je vezana za CPU bez prolaska kroz Interconnect, što odgovara realnoj PS DDR magistrali na Zybo-u. Kapaciteta je 1024 floata (4 KB), organizovana kao kružni bafer za sekvencijalno upisivanje rezultata uzorak po uzorak.
- **Interconnect** - centralna magistrala koja rutira TLM transakcije od CPU-a ka odgovarajućem targetu na osnovu adrese. Ima tri targeta: BRAM (0x0000–0x01FF), AHRS IP (0x0200–0x020F) i SPI kontroler (0x0300–0x031F). Odgovara AXI Interconnect IP-u u Vivado block designu.
- **BRAM** - deljeni međubafer između CPU-a i AHRS IP akceleratora. Kapaciteta je 512 reči × 32-bita (2 KB), dualni port - port A koristi CPU/Interconnect, port B direktno AHRS IP. Za svaki uzorak CPU upisuje jedan HwInputSample u ulazni region, a AHRS IP upisuje jedan HwOutputSample u izlazni region. Regioni su prikazani u Tabeli 1. Modelovan je sa kašnjenjem od 10 ns (1 taktni ciklus na 100 MHz) prema Xilinx UG473.
- **AHRS IP** - hardverski akcelerator koji izvršava jezgro Madgwick algoritma (sekcije 5–8) u fixed-point aritmetici. Prima START signal od CPU-a kroz Interconnect, čita jedan ulazni uzorak iz BRAM-a, obrađuje ga i upisuje nenormalizovani kvaternion nazad u BRAM. Po završetku obrade jednog uzorka šalje interrupt prema CPU-u putem globalnog sc_event ahrs_done. Latencija je 975 ns (65 taktnih ciklusa × 15 ns clock = 66,7 MHz) iz HLS sinteze.

Region	Adresni opseg	Sadržaj
`BRAM_RAW_OFFSET`	0x0020 – 0x005B	Sirovi senzorski podaci (SPI → BRAM): 1 uzorak × 6 reči (gx, gy, gz, ax, ay, az)
`BRAM_INPUT_OFFSET`	0x005C – 0x00C9	HW ulazni bafer (CPU → BRAM): 1 uzorak × 15 reči (HwInputSample)
`BRAM_OUTPUT_OFFSET`	0x00CA – 0x00F1	HW izlazni bafer (IP → BRAM): 1 uzorak × 4 reči (nenormalizovani kvaternion qw, qx, qy, qz)

Tabela 5. Organizacija BRAM memorijskog prostora

5.1 Funkcionisanje virtuelne platforme

Virtuelna platforma funkcioniše kao **petlja akvizicije i obrade** koja se ponavlja za svaki pristigli uzorak. Tok izvršavanja je sledeći:

- **Akvizicija podataka (SPI -> interni bafer):** SPI kontroler svake 10 ms (frekvencija uzorkovanja 100 Hz) inicira TLM READ transakciju prema Sensor modulu i čita jedan sirovi uzorak od 6 floata (gx, gy, gz, ax, ay, az). Uzorak se čuva u internom registru SPI-ja, nakon čega SPI generiše događaj sensor_ready.
- **SW faza 1 - sekcije 1–4 (CPU):** CPU se budi na sensor_ready događaj. Čita sirovi uzorak iz SPI registara (kroz Interconnect), izvršava softverske sekcije 1–4 Madgwick algoritma (provjera opsega žiroskopa, rampanje pojačanja, izračun halfGravity iz tekućeg kvaterniona) i formira HwInputSample strukturu od 15 floata. HwInputSample se upisuje u BRAM_INPUT_OFFSET regiju kroz Interconnect.
- **Pokretanje hardverskog akceleratora (CPU -> AHRS IP):** CPU upisuje vrijednost 1 u kontrolni registar AHRS IP-a (IP_ADDR_OFFSET + IP_REG_CTRL) čime postavlja START bit. AHRS IP detektuje start_event notifikaciju i prelazi u stanje obrade.
- **HW faza - sekcije 5–8 (AHRS IP):** AHRS IP čita jedan HwInputSample iz BRAM-a (port B, direktna veza), izvršava sekcije 5–8 Madgwick algoritma u fixed-point aritmetici (normalizacija akcelerometra, povratna sprega, konverzija žiroskopa, integracija kvaterniona) i rezultujući nenormalizovani kvaternion (4 floata) upisuje u BRAM_OUTPUT_OFFSET regiju. Po završetku AHRS IP notifikuje CPU putem ahrs_done događaja. Latencija obrade jednog uzorka modelovana je kao 975 ns.
- **SW faza 2 - sekcije 9–11 (CPU):** CPU se budi na ahrs_done interrupt. Čita nenormalizovani kvaternion iz BRAM_OUTPUT_OFFSET-a, primenjuje normalizaciju (fast inverse square root, SW sekcija 9), korekciju nultog kursa (sekcija 10) i ažurira interni float kvaternionski stanje (sekcija 11). Normalizovani kvaternion se upisuje u DDR memoriju na tekuću poziciju u kružnom baferu.

Opisani ciklus se ponavlja za svaki od 200 uzoraka iz test skupa. Po obradi poslednjeg uzorka CPU poziva sc_stop() i simulacija se završava.

5.2 AHRS IP modul

AHRS IP je **hardverski akcelerator** koji realizuje sekcije 5–8 Madgwick AHRS algoritma u fixed-point aritmetici. Odgovara HLS generisanom IP core-u koji bi se u realnoj implementaciji uvezao u Vivado block design na Zybo Z7-10 ploči.

Modul sadrži dva TLM socketa: **tsock_interconnect** (target socket) za prijem kontrolnih write/read transakcija od CPU-a kroz Interconnect, i **isock_bram** (initiator socket) za direktan pristup BRAM-u radi čitanja ulaznih uzoraka i upisa rezultata. Interrupt prema CPU-u realizovan je globalnim SystemC događajem ahrs_done koji AHRS IP notifikuje po završetku obrade batch-a, a CPU blokira na wait(ahrs_done).

Tok rada AHRS IP modula je sledeći: modul blokira u SC_THREAD petlji na start_event. Kada CPU upiše START bit u kontrolni registar, b_transport handler notifikuje start_event. SC_THREAD se budi, inkrementira interni brojač uzoraka, čita 15 floata jednog HwInputSample iz BRAM_INPUT_OFFSET-a, poziva internu funkciju hw_sections5to8() koja implementuje algoritam u sc_fixed tipu, i dobijeni nenormalizovani kvaternion (4 floata) upisuje u BRAM_OUTPUT_OFFSET. Nakon toga poziva ahrs_done.notify() i vraća se u čekanje na sledeći start_event.

Interna fixed-point implementacija koristi tipove definisane u ahrs_ip.hpp na osnovu bitske analize. Tipovi su prikazani u Tabeli 2.

Alias	Tip (ukupno, celi deo)	Namena
quat_t	sc_fixed<18, 2>	Kvaternionske komponente [-2, 2)
dot_t	sc_fixed<26, 1>	Skalarni proizvod [-1, 1)
norm_t	sc_fixed<26, 2>	Normalizovano ubrzanje / cross
accel_t	sc_fixed<20, 3>	Ulazni akcelerometar [-4, 4)
hg_t	sc_fixed<26, 2>	halfGravity vektor [-2, 2)
halfgyro_t	sc_fixed<22, 2>	Polu-ugaona brzina [-2, 2)
gain_t	sc_ufixed<20, 4>	Pojačanje rampedGain [0, 16)
dq_t	sc_fixed<27, 5>	Priraštaj kvaterniona
dt_t	sc_ufixed<20, 0>	Vremenski korak deltaTime [0, 1)
qw	sc_fixed<18, 2>	Skalarna komponenta kvaterniona orijentacije iz prethodne iteracije
qx	sc_fixed<18, 2>	x komponenta vektorskog dela kvaterniona iz prethodne iteracije
qy	sc_fixed<18, 2>	y komponenta vektorskog dela kvaterniona iz prethodne iteracije
qz	sc_fixed<18, 2>	z komponenta vektorskog dela kvaterniona iz prethodne iteracije

Tabela 6. Fixed-point tipovi AHRS IP modula

Komponente qw, qx, qy, qz u ulaznoj strukturi predstavljaju normalizovani kvaternion koji CPU šalje pre svake invocacije AHRS IP-a kao reprezentaciju kvaternionskog stanja iz prethodne iteracije. AHRS IP koristi ovaj kvaternion kao operand u sekciji 8 (integracija kvaterniona) jer interno ne čuva stanje između poziva.

Registarska mapa AHRS IP modula prikazana je u Tabeli 3. Sve adrese su relativne u odnosu na baznu adresu IP_ADDR_OFFSET = 0x0200.

Naziv registra	Širina	Opis
IP_REG_CTRL(0x0000)	32-bit	Kontrolni registar. Bit[0]=START: upisom 1 CPU pokreće obradu jednog batch-a. Bit[1]=RESET, Bit[2]=MODE (0=batch).
IP_REG_STATUS(0x0001)	32-bit	Statusni registar (read-only). Bit[0]=BUSY: IP je u obradi. Bit[1]=DONE: obrada završena. Bit[2]=ERROR.
IP_REG_SAMPLE_SIZE(0x0002)	32-bit	Broj reči ulaznog uzorka (INPUT_SAMPLE_WORDS = 15).
IP_REG_SAMPLE_CNT(0x0003)	32-bit	Brojač obrađenih uzoraka od zadnjeg reseta — koristi se za debagovanje.
IP_REG_ERROR_CODE(0x0004)	32-bit	Kôd greške, validan samo kad je STATUS[2]=1.

Tabela 7. Registarska mapa AHRS IP modula

6. Mešovita simulacija

Kosimulacija sprovedena u okviru ovog projekta nalazi se u folderu *esl/cosim/* i pokrenuta je u Cadence Xcelium 19.03 simulatoru komandom *xmsc_run*. Sve sekcije algoritma, uključujući i hardverske sekcije 5-8, modelovane su u SystemC, u *sc_fixed* aritmetici. VHDL implementacija akceleratora verifikovana je nezavisno, kroz RTL simulaciju u Vivadu nad pripadajućim testbenčovima, a izlazi obe simulacije porede se nad istim ulaznim dataset-om.

6.1 Razlozi za odstupanje od klasičnog pristupa

Hardverski akcelerator opisan u *ahrs_ip_v3_0.vhd* i upakovan kroz AXI Lite wrapper interno koristi nekoliko Xilinx Floating-Point operator core-ova iz Vivado kataloga, instance tipa *fp_mul* i *fp_add* za 32-bitne IEEE 754 operacije, kao i konverzione module za prelaz između floating-point i fixed-point reprezentacija. Te komponente nisu standardni VHDL fajlovi, već generisani IP-evi čije funkcionalne definicije postoje u Xilinx simulacionim bibliotekama.

Cadence Xcelium ne prepoznaje te biblioteke u standardnoj instalaciji, one moraju biti unapred kompajlirane specifično za Xcelium kao ciljni simulator i povezane sa simulacionim okruženjem preko konfiguracionih fajlova *cds.lib* i *hdl.var*. Instalacija Xcelium 19.03 na školskom serveru kojem imamo pristup ne sadrži te biblioteke, a na istoj mašini nije dostupan Vivado iz koga bi se one generisale. Klasična mešovita simulacija sa instanciranim *ahrs_ip_v3_0* entity-jem zbog toga nije izvodljiva u datim uslovima.

Entity *ahrs_sec678*, koji implementira samo sekcije 6, 7 i 8 u čistoj fixed-point aritmetici i ne zavisi od FP IP core-ova, ne predstavlja smislenu zamenu, pokriva tri od četiri hardverske sekcije, dok sekcija 5 sadrži operacije inverznog kvadratnog korena i deljenja koje su u RTL verziji realizovane upravo kroz FP IP-eve.

6.2 Realizacija kosimulacije

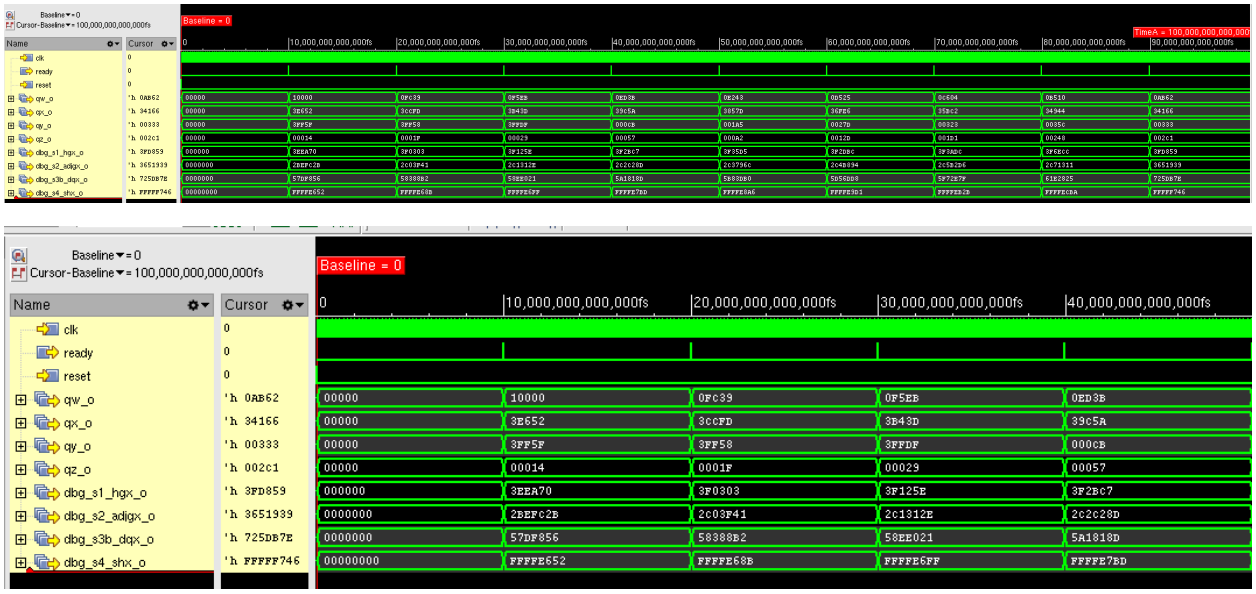
Struktura kosimulacije ponavlja strukturu virtuelne platforme iz *esl/vp/*. Sedam modula, SENSOR, SPI, CPU, BRAM, INTERCONNECT, AHRS_IP i DDR - povezani su TLM-2.0 socketima, sa identičnim memorijskim mapiranjem i istim sinhronizacionim događajima (*sensor_ready*, *ahrs_done*). Modul AHRS_IP izvršava sekcije 5-8 u *sc_fixed* reprezentaciji sa Q formatima koji odgovaraju onima u VHDL implementaciji: kvaternioni u Q2.16 (*sc_fixed<18, 2, SC_RND, SC_SAT>*), polu-gravitacioni vektor u Q2.24 (*sc_fixed<26, 2>*), ramped gain u Q4.16 (*sc_ufixed<20, 4>*), vremenski korak u Q0.20 (*sc_ufixed<20, 0>*).

Razlika u odnosu na samostalnu virtuelnu platformu iz *esl/vp/* je u build okruženju. Platforma iz *esl/vp/* se kompajlira kroz *g++* uz lokalnu SystemC instalaciju i pokreće kao samostalan izvršni fajl. Kosimulacija iz *esl/cosim/* se kompajlira i pokreće kroz *xmsc_run* u Xcelium okruženju, sa istim parametrima koji bi važili i u slučaju da je dodat VHDL DUT - što daje pristup SimVision alatu, talasnim formama svih signala i isti tok rada koji prati standardnu mešovitu simulaciju.

Komanda za pokretanje navedena je u skripti *pokreni.sh*:

```
xmsc_run -64bit -sc_main -gui -access +rwc \  -DSC_INCLUDE_FX
-DSC_INCLUDE_DYNAMIC_PROCESSES \  -Wcxx,-std=c++17 \  sc_main.cpp ahrs_ip.cpp
cpu.cpp bram.cpp spi.cpp \  sensor.cpp ddr.cpp interconnect.cpp
```

Tok jedne simulacione iteracije odvija se na sledeći način. SENSOR modul po pokretanju učitava sve uzorke iz CSV fajla *data1504.csv*. SPI kontroler ih svakih 10 ms predaje virtuelnoj magistrali. CPU obrađuje sekcije 1-4, formira strukturu *HwInputSample* i upisuje je u BRAM, zatim postavlja START bit u kontrolnom registru akceleratora i čeka prekid. AHRS_IP izvršava sekcije 5-8 i upisuje rezultujući kvaternion u BRAM, posle čega emituje *ahrs_done* događaj. CPU čita kvaternion, normalizuje ga u sekcijama 9-11 i upisuje konačnu orijentaciju u DDR. Postupak se ponavlja za svih 1918 uzoraka iz dataset-a, a izlazni kvaternioni se po završetku simulacije ispisuju u *vp_output.csv*.



Slika 7. Izgled waveform-a nakon pokrenute simulacije

```
[SC_MAIN] Starting simulation...

xcelium>
xcelium> run
[SPI] Sample 1 ready. Notifying CPU...

[CPU] *** Sample 1/10 ***
[CPU] Sample 1 done. q=(0.995231, -0.0998342, -0.0023904, -0.000240039)
[SPI] Sample 2 ready. Notifying CPU...

[CPU] *** Sample 2/10 ***
[CPU] Sample 2 done. q=(0.980379, -0.198283, -0.00235903, -0.000477576)
[SPI] Sample 3 ready. Notifying CPU...

[CPU] *** Sample 3/10 ***
[CPU] Sample 3 done. q=(0.955897, -0.29449, -0.000282366, -8.73512e-05)
[SPI] Sample 4 ready. Notifying CPU...

[CPU] *** Sample 4/10 ***
[CPU] Sample 4 done. q=(0.922174, -0.387358, 0.00309185, 0.00129872)
[SPI] Sample 5 ready. Notifying CPU...

[CPU] *** Sample 5/10 ***
[CPU] Sample 5 done. q=(0.879548, -0.476247, 0.00597326, 0.00323479)
[SPI] Sample 6 ready. Notifying CPU...

[CPU] *** Sample 6/10 ***
[CPU] Sample 6 done. q=(0.82857, -0.560196, 0.00875886, 0.00592277)
[SPI] Sample 7 ready. Notifying CPU...

[CPU] *** Sample 7/10 ***
[CPU] Sample 7 done. q=(0.769774, -0.638525, 0.0106929, 0.00887115)
[SPI] Sample 8 ready. Notifying CPU...

[CPU] *** Sample 8/10 ***
[CPU] Sample 8 done. q=(0.703893, -0.710459, 0.0109029, 0.0110065)
[SPI] Sample 9 ready. Notifying CPU...

[CPU] *** Sample 9/10 ***
[CPU] Sample 9 done. q=(0.668609, -0.743638, 0.0109208, 0.0121473)
[SPI] Sample 10 ready. Notifying CPU...

[CPU] *** Sample 10/10 ***
[CPU] Sample 10 done. q=(0.632632, -0.77445, 0.0109397, 0.0133931)

[CPU] Svih 10 uzoraka obradeno. Zaustavljam simulaciju.
SystemC: simulation stopped by user.


SC simulation stopped by user.

SystemC : SystemC stopped at time 100000830
```

Slika 8. Ispis za prvih 10 sample-ova nakon pokretanja cosimulacije

6.3 Korišćeni fajlovi

Folder *esl/cosim/* sadrži izvorne fajlove sedam modula virtuelne platforme (*sc_main.cpp*, *ahrs_ip.cpp/hpp*, *cpu.cpp/hpp*, *bram.cpp/hpp*, *spi.cpp/hpp*, *sensor.cpp/hpp*, *ddr.cpp/hpp*, *interconnect.cpp/hpp*), zaglavlje sa zajedničkim definicijama (*defines.hpp*) i skriptu *pokreni.sh* za pokretanje simulacije.

Ulazni podaci se učitavaju iz *../../data/data1504.csv* (1918 uzoraka IMU senzora), a po završetku simulacije DDR modul ispisuje izlazne kvaternione u *vp_output.csv* u istom formatu kao i samostalna virtuelna platforma, što omogućava direktno poređenje sa ESL referentnim modelom i sa rezultatima RTL simulacije.

6.4 Verifikaciona pokrivenost

Kosimulacija u Xcelium okruženju verifikuje sistemski nivo virtuelne platforme - tok podataka između modula, mapiranje memorije, semantiku AXI Lite kontrolnih registara akceleratora i sinhronizaciju procesora i akceleratora preko prekida.

Izlazni kvaternioni iz kosimulacije porede se sa izlazima RTL simulacije sprovedene u Vivadu nad testbenčovima *tb_ahrs_sec678.vhd* i *tb_ahrs_sec5to8.vhd*, nad istim *data1504.csv* dataset-om. Podudaranje izlaza zadržava se kroz svih 10 sekundi rada algoritma.

Tajming-zavisne karakteristike hardvera - pet ciklusa pipeline latencije akceleratora, latencije FP IP-ova od tri i četiri ciklusa, kritični putovi nakon sinteze - nisu predmet ove kosimulacije i pokrivene su RTL simulacijom i analizom posle sinteze u Vivadu.

7. Procena performansi

Tabela ispod prikazuje vrednosti dobijene iz Vitis HLS izveštaja C-sinteze i Vivado izveštaja sinteze za AHRS IP jezgro na platformi Zybo Z7-10 (xc7z010clg400-1).

Parametar	Vrednost	Izvor
Tajming i propusna moć		
Ciljni uređaj	**xc7z010clg400-1**	HLS / Vivado
Period takta	**15 ns**	HLS sinteza
Taktna frekvencija	**66,7 MHz**	1 / 15 ns
Latencija po uzorku	**65 cik. = 975 ns**	HLS izveštaj
Slack	0,42 ns	HLS izveštaj
Maks. throughput	**~1 026 000 uzoraka/s**	1 / 975 ns
Zahtev senzora	100 Hz (10 ms/uzorku)	Specifikacija
Faktor ubrzanja vs. zahtev	**~10 256×**	10 ms / 975 ns
Resursi (Vivado sinteza)		
DSP48E1	**78 / 80 (97,5%)**	Vivado sinteza
LUT (Slice Logic)	**9 738 / 17 600 (55,3%)**	Vivado sinteza
Flip Flop (FDRE/FDSE)	**4 455 / 35 200 (12,7%)**	Vivado sinteza
BRAM	0 / 60 (0%)	Vivado sinteza

Tabela 8. Performanse i resursi AHRS IP jezgra

Napomena o LUT proceni: HLS C-sinteza procenila je 18 158 LUT (103%), što je iznad kapaciteta uređaja. Međutim, Vivado RTL sinteza sa optimizacijama dala je 9 738 LUT (55,3%). HLS procena je konzervativna i ne uzima u obzir logičke optimizacije koje Vivado primenjuje tokom implementacije.

7.1 Postupak HLS-a

Prilikom pokretanja HLS analize prvi put, iskorišćenost DSP blokova prelazila je 200%, pa je originalna verzija, u kojoj se radilo obrađivanje u batch-evima od po 5 sample-ova, zamenjena novom verzijom koda, u kojoj se obrađuje sample-po-sample.

Ponovno pokretanje HLS analize nije odmah dalo dobre rezultat, pa se uz par izmena, iskorišćenost DSP blokova i LUT-ova smanjila.
Problem je bio što HLS instancira fast_inv_sqrt dva puta kao posebne hardverske blokove. Možemo ga prisiliti da deli jednu hardversku instancu za oba poziva:
DSP usage od fast_inv_sqrt se prepolovljava jer oba poziva dele isti HW blok.

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	940.000	95	no	function	0	95	8,582	18,388	0
fast_inv_sqrt	200.000	20	no	function	0	5	560	917	0

Slika 9. Rezultati HLS-a u Vitisu

Koristimo 95 DSP blokova, a na raspolaganju imamo samo 80. Pored toga, broj LUT-ova takođe prelazi granicu.

Nakon toga, iskorišćenost resursa još uvek nije bila zadovoljavajuća, pa je odlučeno da se pokuša sa smanjenjem broja bitova samo za intermedijarne tipove:

- dot_t: 26 -> 18 bita – koristi se samo za poređenje (if dot < 0), ne za dalje računanje
- magsq_t: 22 -> 18 bita – ulaz u fast_inv_sqrt, malo utiče na krajnji rezultat
- invmag_t: 24 -> 18 bita – intermedijaran, rezultat se odmah kasti na norm_t

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	940.000	95	no	function	0	92	8,536	18,200	0
fast_inv_sqrt	200.000	20	no	function	0	5	560	917	0

Slika 10. Rezultati HLS-a u Vitisu nakon smanjenja broja bitova za određene tipove

Obzirom da rezultati pokazuju da je iskorišćenost još uvek velika, uvode se određene pragme koje bi trebalo da pomognu. Prva pragma koja je uvedena jeste ALLOCATION operation.

ALLOCATION operation ograničava ukupan broj množilica i primorava HLS da ih time-multipleksira kroz manje DSP instanci.
Ključna nova pragma je:

#pragma HLS ALLOCATION operation instances=mul limit=30

Ovo primorava HLS da sve množenja rasporedi kroz 30 DSP instanci umesto ~90 paralelnih.

```
+ Performance & Resource Estimates:

    PS: '+' for module; 'o' for loop; '*' for dataflow
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
    |      Modules     | Issue|      |      Latency      | Iteration|         | Trip |          |      |           |           |              |     |
    |      & Loops     | Type | Slack| (cycles)|   (ns)  |  Latency | Interval| Count| Pipelined| BRAM |    DSP    |     FF    |      LUT     | URAM|
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
    |+ ahrs_ip_hls     |     -|  0.04|       94|  940.000|         -|       95|     -|        no|     -|  86 (107%)|  7678 (21%)|  18453 (104%)|    -|
    | + fast_inv_sqrt  |     -|  0.04|       20|  200.000|         -|       20|     -|        no|     -|     5 (6%)|   560 (1%)|    917 (5%)|    -|
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
```

Slika 11. Rezultati HLS-a u Vitisu nakon uvođenja ALLOCATION operation pragme

Rezultati još uvek nisu bili sasvim odgovarajući, pa je sledeća ideja bila da se poveća clk sa 10ns na 15ns, HLS time dobija više slack-a i može da ulančava operacije kroz više ciklusa umesto da instancira paralelne DSP kopije.

Timing Estimate

TARGET	ESTIMATED	UNCERTAINTY
15.00 ns	10.533 ns	4.05 ns

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	975.000	66	no	function	0	86	5,310	18,245	0
fast_inv_sqrt	225.000	15	no	function	0	5	562	897	0

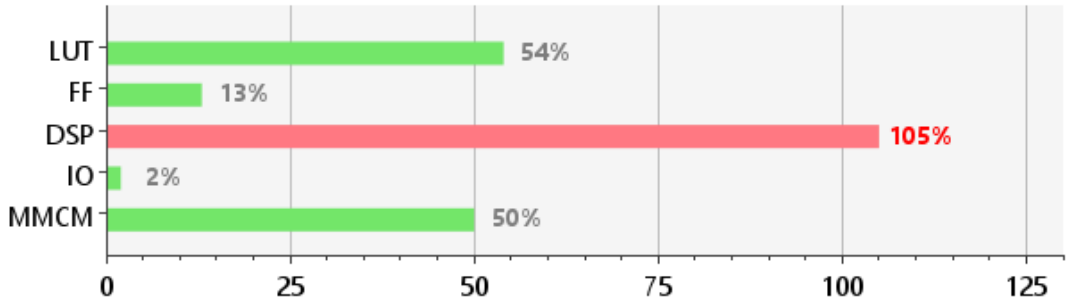
Slika 12. Rezultati HLS-a u Vitisu nakon promene clk na 15ns

Vivado implementacija ponekad daje bolje rezultate od HLS procene jer radi fizičko mapiranje na čip. HLS procena je konzervativna. Postoji šansa da čak i sa 86 DSP procenom, Vivado implementacija stvarno stane u 80 DSP. Zbog toga je sledeći korak bio da se eksportuje IP iz HLS-a i da se uvuče u Vivado projekat, pa da se tamo odradi sinteza. Korišćen je identičan kod, kao kod na osnovu kog su dobijeni rezultati sa slike 4.

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	DSPs (80)	Bonded IOB (100)	BUFGCTRL (32)	MMCME2_ADV (2)
ahrs_bd	9557	4417	30	84	2	2	1
ahrs_ip_hls_0 (ahrs_bd_ahrs_ip_hls_0_0)	9538	4377	30	84	0	0	0

Slika 13. Rezultati sinteze u Vivadu

Resource	Utilization	Available	Utilization %
LUT	9557	17600	54.30
FF	4417	35200	12.55
DSP	84	80	105.00
IO	2	100	2.00
MMCM	1	2	50.00



Slika 14. Grafički prikaz rezultata na osnovu sinteze odrađene u Vivadu

Broj DSP blokova se smanjio, ali nedovoljno, a broj LUT-ova je čak značajno manji, nego što je bio prilikom pokretanja HLS-a u Vitisu. Zbog toga će, u daljem radu, biti nastavljeno pokretanje i sinteze u Vivadu, kako bi se došlo do što realističnijih rezultata.

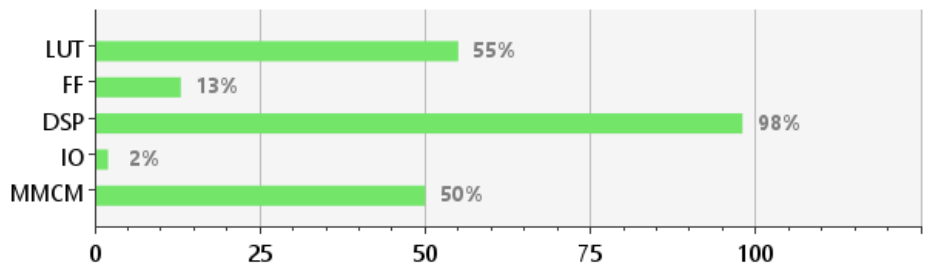
Rezultati su bili jako blizu granice, te je pokušano smanjenje limita u pragmi sa 30 na 26:

#pragma HLS ALLOCATION operation instances=mul limit=26

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	F8 Muxes (4400)	DSPs (80)	Bonded IOB (100)	BUFGCTRL (32)	MMCME2_ADV (2)
ahrs_bd_wrapper	9757	4495	31	1	78	2	2	1
ahrs_bd_i (ahrs_bd)	9757	4495	31	1	78	0	2	1

Slika 15. Rezultati sinteze u Vivadu nakon smanjenja limita pragme

Resource	Utilization	Available	Utilization %
LUT	9757	17600	55.44
FF	4495	35200	12.77
DSP	78	80	97.50
IO	2	100	2.00
MMCM	1	2	50.00



Slika 16. Grafički prikaz rezultata na osnovu sinteze odrađene u Vivadu nakon smanjenja limita pragme

Nakon dobijanja zadovoljavajućih rezultata, pokrenut je i HLS u Vitisu, kako bi se dobijene vrednosti potvrdile.

Estimated Quality of Results

Timing Estimate

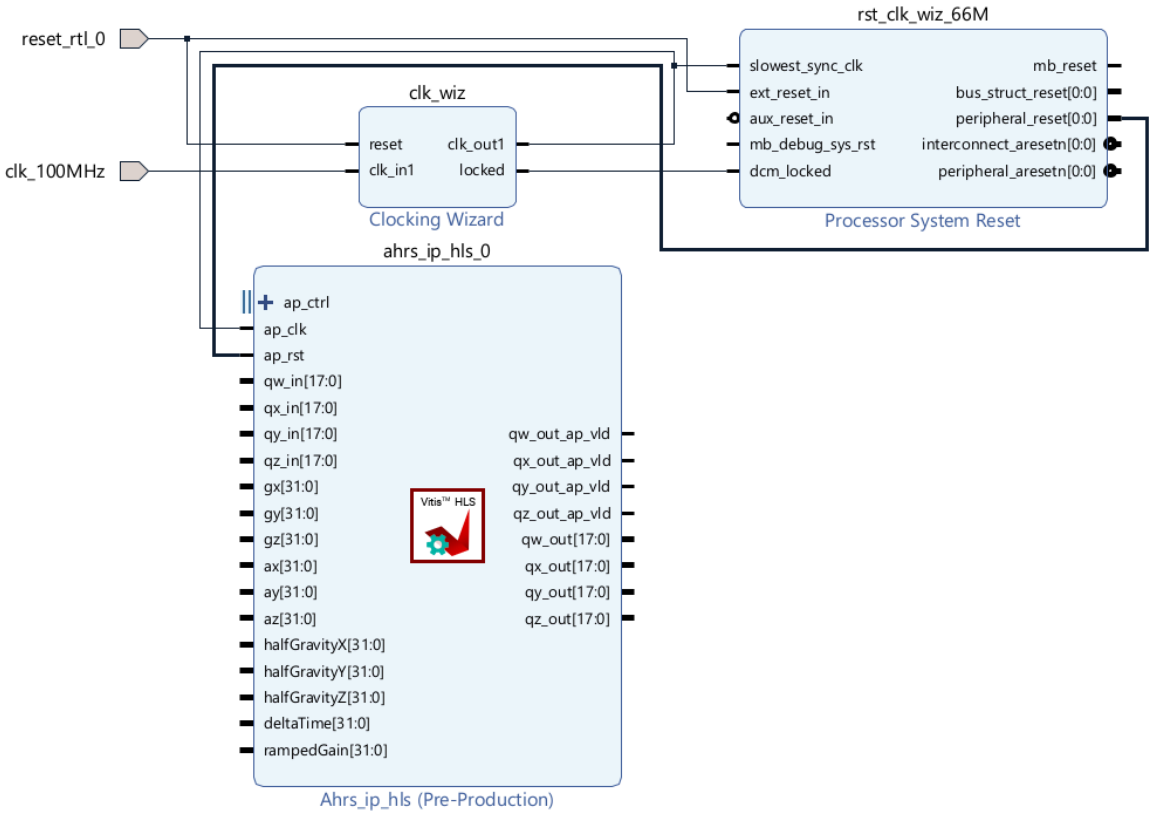
TARGET	ESTIMATED	UNCERTAINTY
15.00 ns	10.533 ns	4.05 ns

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	975.000	66	no	function	0	78	5,320	18,158	0
fast_inv_sqrt	225.000	15	no	function	0	5	562	897	0

Slika 17. Pokretanje HLS-a u Vitisu



Slika 18. Izgled IP bloka u Vivadu

Rezultat je uspešno dokazao da je broj iskorišćenih DSP blokova u granicama u odnosu na dostupne resurse. Iskorišćenost DSP blokova spala je na 97%, dok je broj iskorišćenih LUT-ova, u odnosu na dostupan broj, jednak 55%.

Bez pragme, HLS instancira sve množilice paralelno – svako množenje dobija svoj DSP48E1 blok koji radi istovremeno sa svim ostalim. To je brže ali troši ~92 DSP-a odjednom.

Sa limit=26, HLS sme da instancira maksimalno 26 množača u isto vreme. Preostalih množenja se raspoređuju sekvencijalno kroz tih 26 instanci – jedno čeka da drugo završi pa koristi isti DSP blok.

Preciznost - nula uticaja. Pragma ne menja ni jedan bit ni jednu operaciju. Isti izračun, isti redosled, isti tipovi. Samo se menja *kada* se koji množilic aktivira, ne *šta* računa.

Brzina - minimalan uticaj. Latencija po uzorku je porasla sa 940ns na 975ns - razlika od 35ns. Na 100Hz mo10ms između uzoraka, a IP završi za ispod 1ms.

8. Literatura

[1] dr Vuk Vranjković, *Projektovanje elektronskih uređaja na sistemskom nivou – računarske vežbe*, Fakultet tehničkih nauka, Univerzitet u Novom Sadu, 2020.

[2] x-io Technologies, "Fusion – Inertial Measurement Library," [Online]. Available: https://github.com/xioTechnologies/Fusion. [datum pristupa: 01-Apr-2025].

[3] Digilent, "PmodNAV Reference Manual," *Digilent Reference*, [Online]. Available: https://digilent.com/reference/pmod/pmodnav/start. [datum pristupa: 01-Apr-2025].

[4] S. O. H. Madgwick, A. J. L. Harrison, R. Vaidyanathan, „Estimation of IMU and MARG orientation using a gradient descent algorithm," IEEE Int. Conf. on Rehabilitation Robotics, Zurich, 2011, str. 1–7.

[5] Xilinx Inc., „Vivado Design Suite User Guide: High-Level Synthesis," UG902 (v2022.2), 2022.

[6] Xilinx Inc., „7 Series FPGAs DSP48E1 Slice User Guide," UG479, 2018.

[7] IEEE, „IEEE Standard for Standard SystemC Language Reference Manual," IEEE Std 1666-2011, 2012.

[8] Xilinx Inc., „Zynq-7000 SoC Technical Reference Manual," UG585, 2021.